



Implémentation des fonctionnalités CRUD avec Entity Framework et ASP.NET MVC

Date de publication : 04 juillet 2011.

Par Jérôme Lambert (Espace perso) (Blog)

[Version PDF \(Miroir\)](#) [Version hors-ligne \(Miroir\)](#)

- I. Introduction
- II. Prérequis techniques
- III. Mise en place de la base de données
- IV. Création du projet Visual Studio 2010
- V. Ajout du modèle ADO.NET Entity Framework
- VI. Développement du site web
 - VI-A. Implémentation de la grille de données
 - VI-B. Implémentation de l'affichage détaillé
 - VI-C. Implémentation de la création
 - VI-D. Implémentation de l'édition
 - VI-E. Implémentation de la suppression
- VII. Conclusion

I. Introduction

Ce tutoriel a pour but de vous expliquer comment développer un site web ASP.NET MVC permettant les opérations CRUD (Create, Read, Update et Delete) au travers d'Entity Framework.

A la fin de ce tutoriel, vous aurez appris comment :

- Créer un projet ASP.NET MVC sous Visual Studio 2010
- Créer et lier un modèle ADO.NET Entity Framework à votre projet
- Développer les écrans web permettant les actions suivantes :
 - - Afficher les données dans une grille provenant d'une table
 - - Afficher le détail d'un élément
 - - Autoriser l'édition d'un élément
 - - Permettre la création d'un nouvel élément
 - - Supprimer un élément existant

II. Prérequis techniques

Afin de reproduire les étapes de ce tutoriel sur votre machine, il vous faudra avoir installé les outils suivants :

- Installer Visual Web Developer 2010 Express
- Installer Visual Studio Service Pack 1
- Installer SQL Server 2008 R2 et Management Tools

III. Mise en place de la base de données

Dans le cadre de ce tutoriel, nous utilisons la base de données " Adventure Works " de Microsoft.

Vous pouvez télécharger cette base de données ainsi que d'autres bases de données d'exemples sur le site Web CodePlex.

AdventureWorks 2008R2 SR1

Rating: ★★★★★ based on 36 ratings

Reviewed: [29 reviews](#)

Downloads: 94258

Released: Nov 17 2010

Updated: Jan 15 2011 by [Gaile MSFT](#)

Dev status: Stable

Downloads

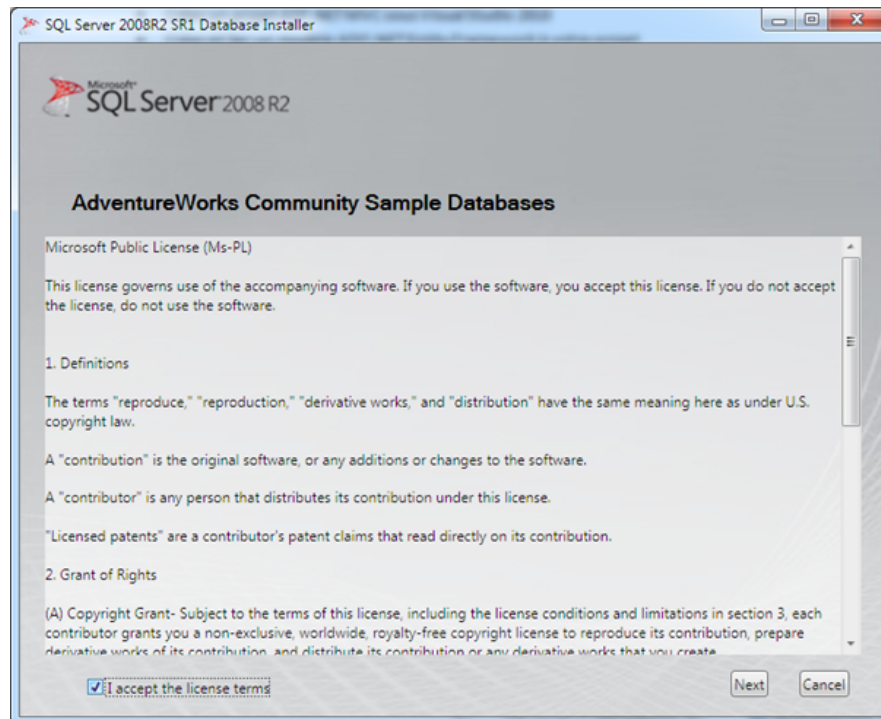


[AdventureWorks2008R2_SR1.exe](#)

application, 84280K, uploaded Dec 2 2010 - 94258 downloads

Une fois le fichier télécharger sur votre machine, lancez l'exécutable afin de démarrer le programme d'installation des bases de données sur votre serveur Microsoft SQL Server.

Acceptez les termes de la licence d'utilisation.

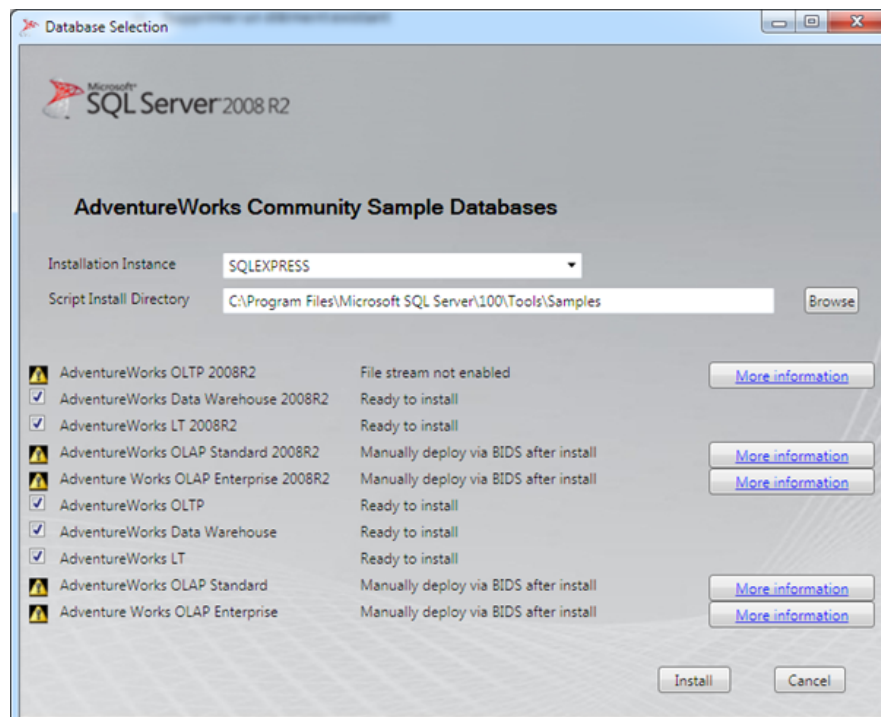


Sélectionnez ensuite votre serveur SQL dans la liste déroulante " Installation Instance ". Si votre serveur SQL n'est pas présent dans la liste, vous pouvez le renseigner.

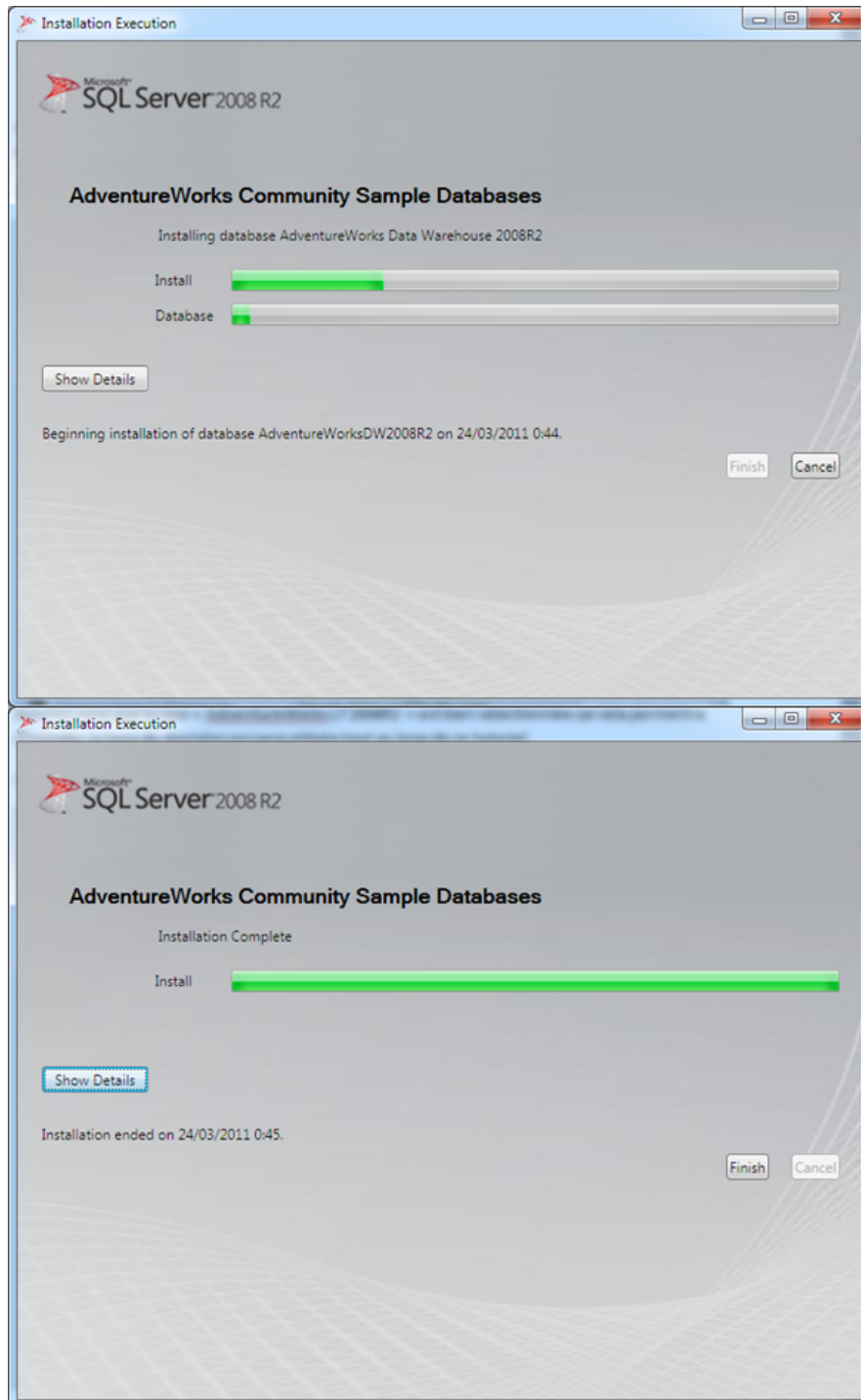
Si vous avez procédé à une installation par défaut de Microsoft SQL Server 2008 R2, le serveur se nomme " SQLEXPRESS ".

Assurez-vous que la case " AdventureWorks LT 2008R2 " est bien sélectionnée car cela permettra d'installer la base de données qui sera utilisée tout au long de ce tutoriel.

Cliquez ensuite sur le bouton " Install ".

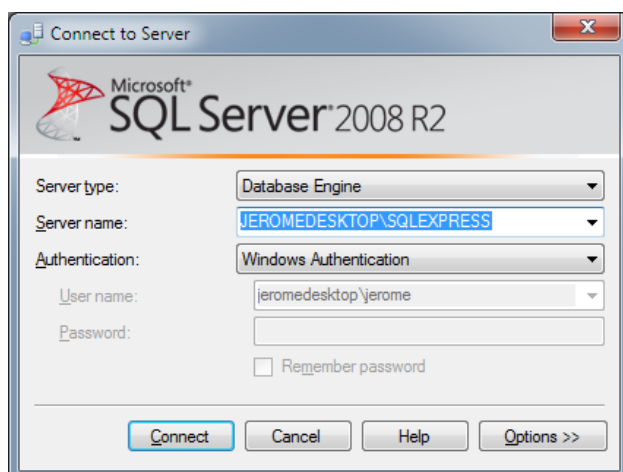


Une fois l'installation terminée, vous pouvez cliquer sur le bouton " Finish ".



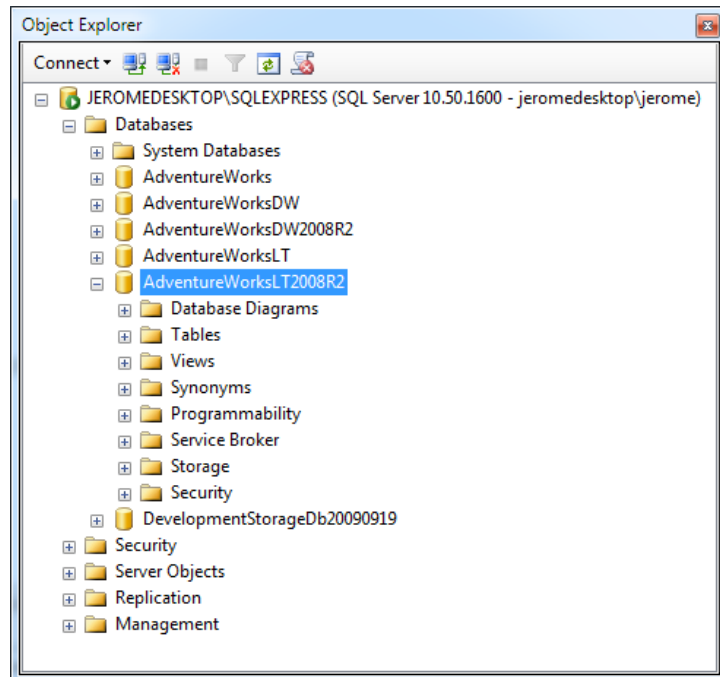
Afin de vérifier que l'installation s'est bien passée, nous allons nous connecter sur le serveur SQL afin de s'assurer que notre base de données est bien présente.

Démarrez " SQL Server Management Studio " et connectez-vous à votre serveur SQL.



Une fois connecté, allez dans la fenêtre " Object Explorer " et ouvrez le nœud " Databases ". Vous devez y retrouver la base de données "

AdventureWorksLT2008R2 " qui est la base de données utilisées tout au long de ce tutoriel.



Nous utiliserons principalement la table " Product ".

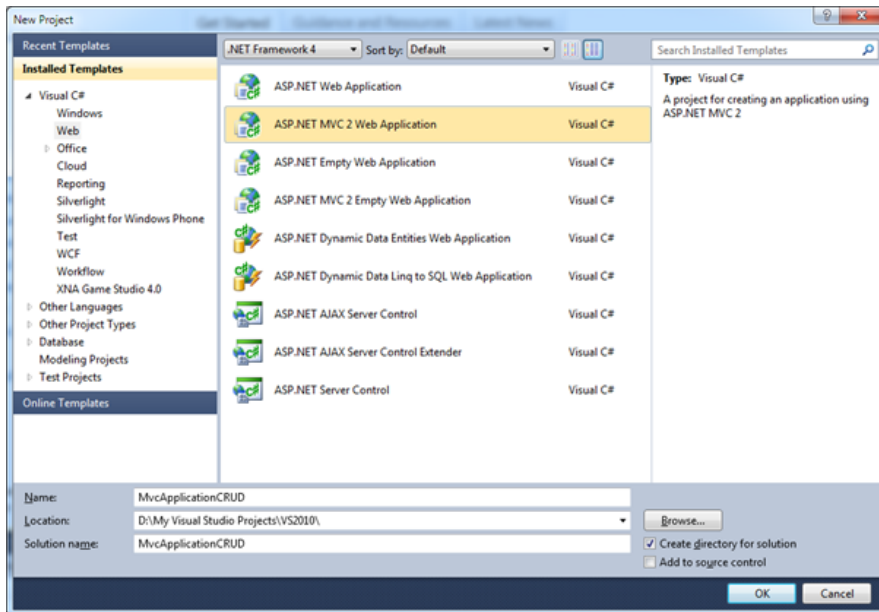
Product (SalesLT)			
	Column Name	Data Type	Allow Nulls
ProductID	ProductID	int	☐
Name	Name	nvarchar(50)	☐
ProductNumber	ProductNumber	nvarchar(25)	☐
Color	Color	nvarchar(15)	☒
StandardCost	StandardCost	money	☐
ListPrice	ListPrice	money	☐
Size	Size	nvarchar(5)	☒
Weight	Weight	decimal(8, 2)	☒
ProductCategoryID	ProductCategoryID	int	☒
ProductModelID	ProductModelID	int	☒
SellStartDate	SellStartDate	datetime	☐
SellEndDate	SellEndDate	datetime	☒
DiscontinuedDate	DiscontinuedDate	datetime	☒
ThumbNailPhoto	ThumbNailPhoto	varbinary(MAX)	☒
ThumbNailPhotoFileName	ThumbNailPhotoFileName	nvarchar(50)	☒
rowguid	rowguid	uniqueidentifier	☐
ModifiedDate	ModifiedDate	datetime	☐

IV. Création du projet Visual Studio 2010

Maintenant que notre base de données est bien installée, nous allons créer notre premier projet ASP.NET MVC sous Visual Web Developer 2010 Express.

Démarrez Visual Web Developer 2010 Express.

Une fois le programme lance, nous allons créer notre projet. Pour cela, allez dans le menu " File\New " et cliquez sur " Project ".

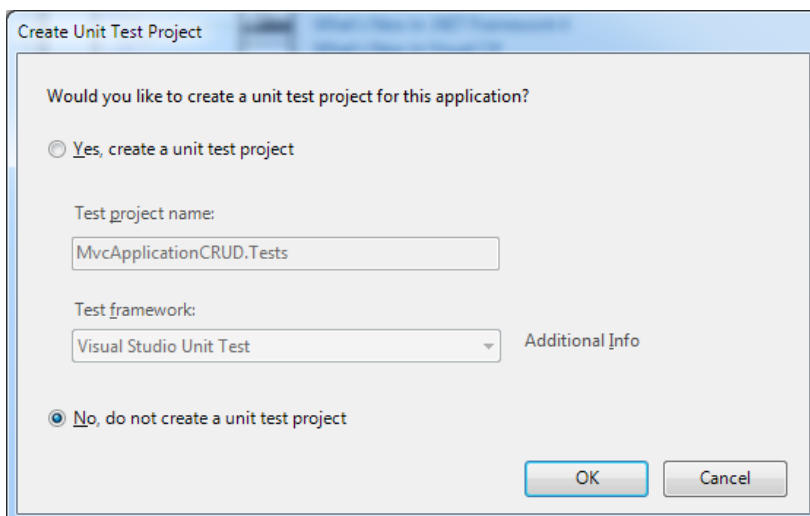


Dans le panneau latéral gauche, sélectionnez la catégorie " Web ".

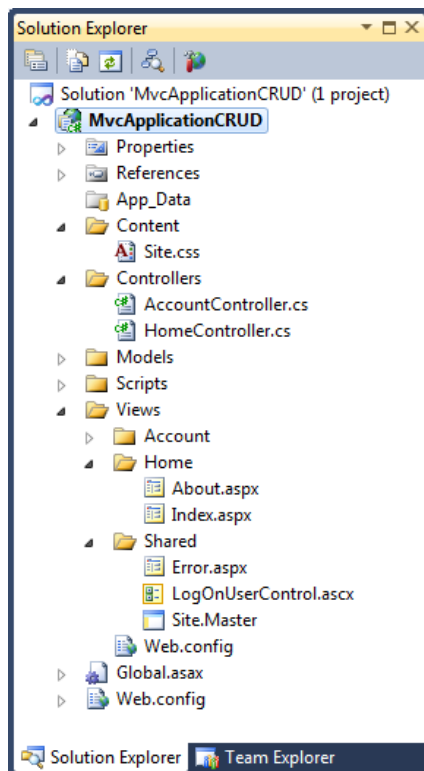
Assurez-vous que la version du .NET Framework est bien la 4.

Sélectionnez ensuite le template " ASP.NET MVC 2 Web Application ", appelez votre projet " MvcApplicationCRUD " et cliquez sur le bouton " OK " pour créer votre projet.

Une boîte de dialogue apparaît vous proposant de créer un projet supplémentaire dédié à la gestion des tests unitaires. Ce point ne faisant pas partie de ce tutoriel, sélectionnez l'option " No, do not create a unit test project. ". Cliquez ensuite sur le bouton " OK ".

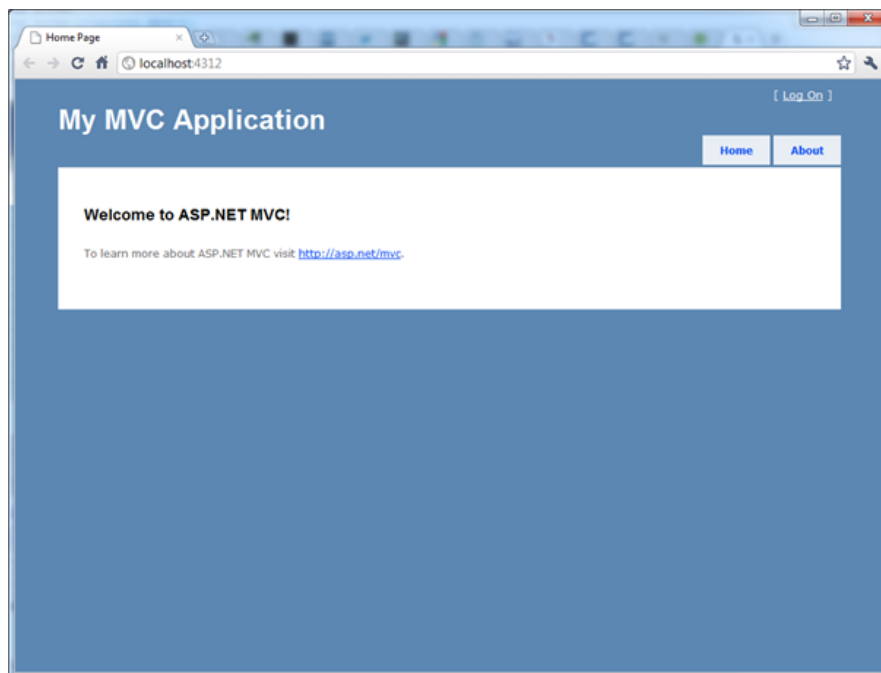


Une fois le projet créé, vous pourrez voir les éléments suivants dans la fenêtre " Solution Explorer " :



Vous pouvez lancer le site Web via le menu " Debug " en cliquant sur " Start Debugging ".

Votre navigateur web par défaut sera lancé automatiquement afin de vous afficher le site web de votre projet.

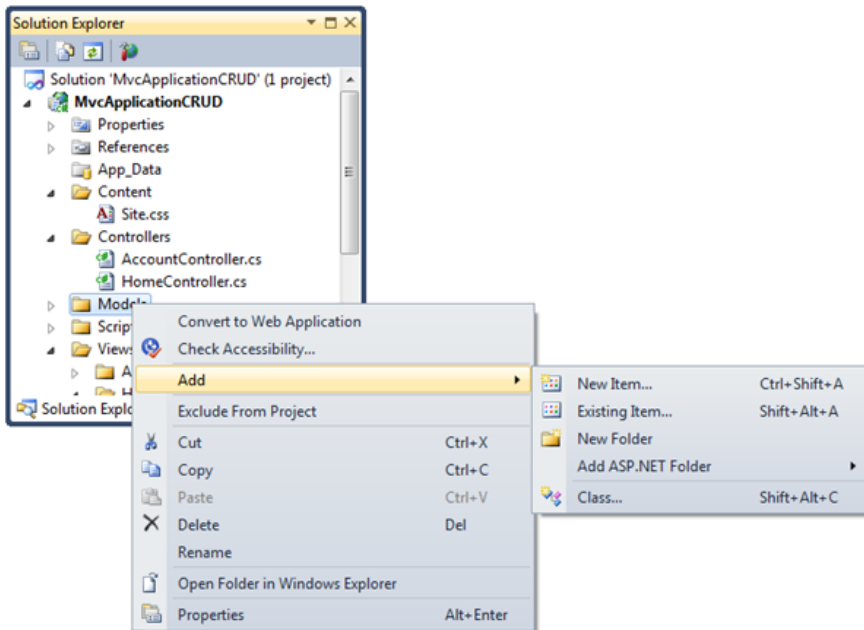


Vous pouvez remarquer que Visual Studio a créé pour vous un template Web.

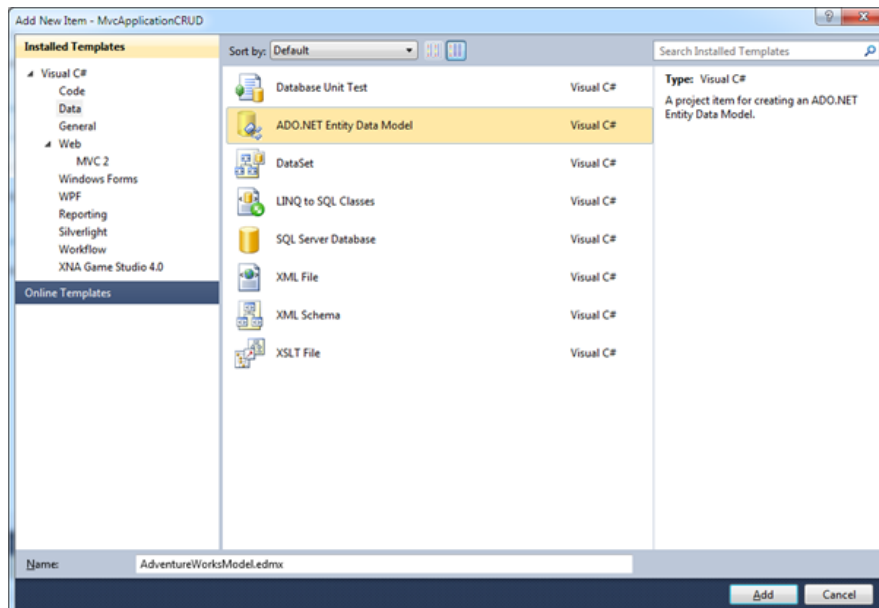
V. Ajout du modèle ADO.NET Entity Framework

Nous allons à présent créer notre modèle ADO.NET Entity Framework qui nous permettra d'interroger la base de données " AdventureWorksLT2008R2 " précédemment créée.

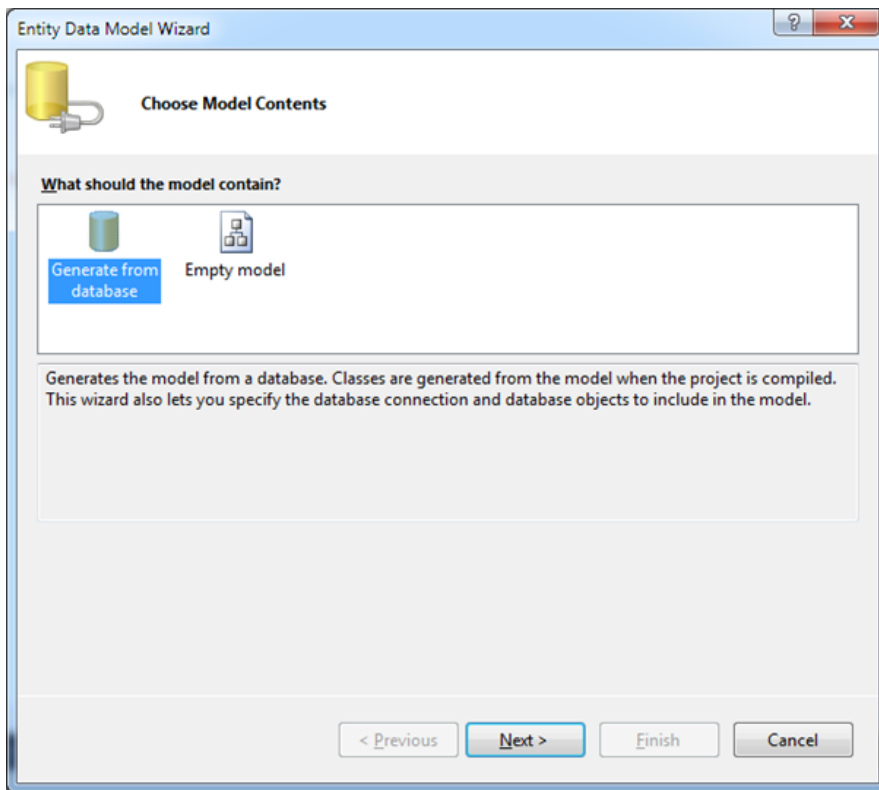
Effectuez un clic droit sur le répertoire " Models " du projet et sélectionnez " Add \ New Item... ".



Une fois la fenêtre affichée, allez dans la catégorie " Data " et sélectionnez " ADO.NET Entity Data Model ". Nommez votre fichier " AdventureWorksModel.edmx " et cliquez sur le bouton " Add ".

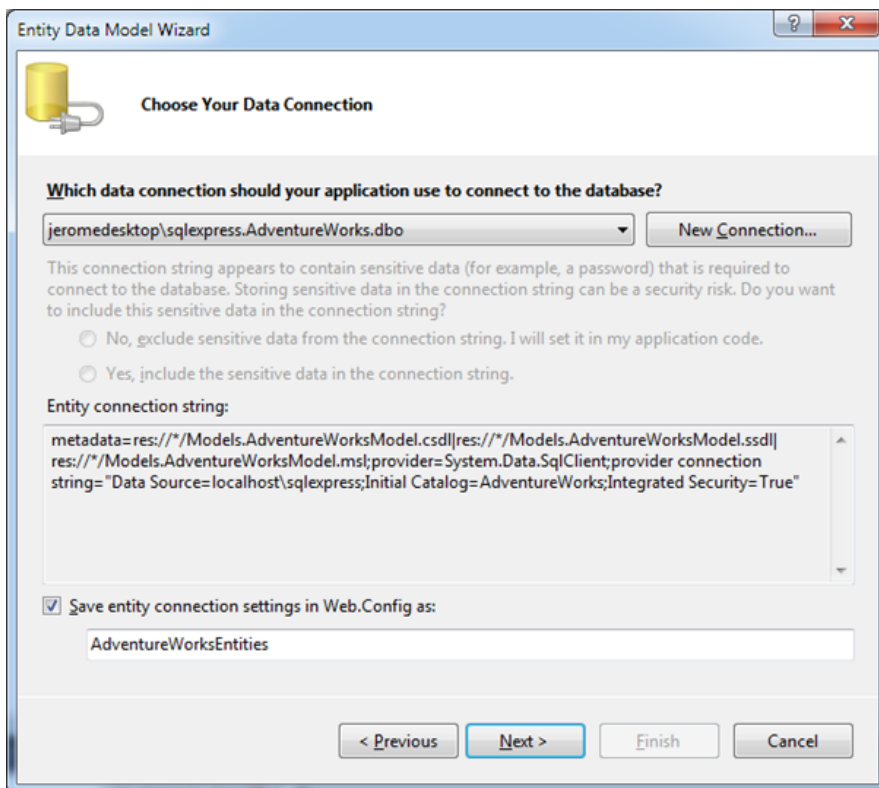


Un wizard va ensuite apparaître pour nous permettre de lier un schéma de base de données à notre modèle. Etant donné que notre base de données existe déjà, nous allons sélectionner l'élément " Generate from database " et cliquer ensuite sur le bouton " Next ".



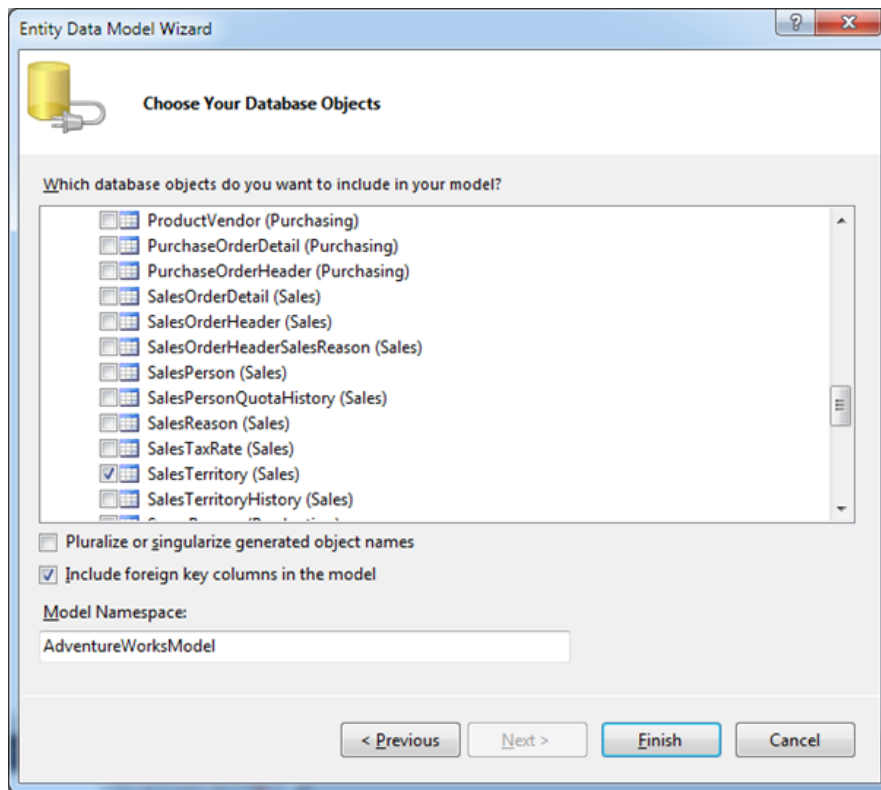
Connectez-vous ensuite à la base de données " AdventureWorks ". Si vous utilisez l'authentification Windows, vous n'avez rien d'autre à faire. Par contre, dans le cas d'une authentification SQL Server, vous devez autoriser la sauvegarde du mot de passe dans la chaîne de connexion ; pour cela, sélectionnez " Yes, include the sensitive data in the connection string. ".

Vous pouvez alors passer à l'étape suivante en cliquant sur le bouton " Next ".

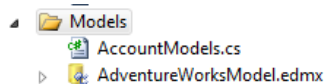


Comme annoncé principalement, le tutoriel se repose sur la table " SalesTerritory ", nous allons donc sélectionner uniquement cette table au sein du schéma. Pour cela, ouvrez le nœud " Tables " et cocher la case " SalesTerritory (Sales) ".

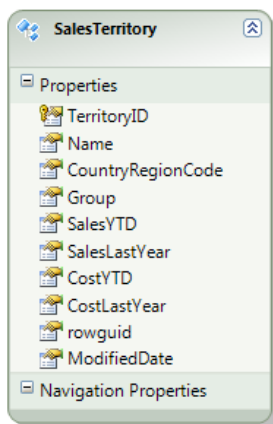
Vous pouvez enfin clôturer le wizard en cliquant sur le bouton " Finish ".



Vous pouvez constater qu'un nouveau fichier " AdventureWorksModel.edmx " a été créé dans notre projet : il s'agit de notre modèle ADO.NET Entity Framework.



Si nous l'ouvrons, nous retrouvons un designer avec une entité " SalesTerritory " mappée à la table " SalesTerritory " de notre base de données.



VI. Développement du site web

Maintenant que notre base de données a été déployée sous SQL Server et que notre projet contient un modèle ADO.NET Entity Framework pour communiquer avec cette base de données, nous allons pouvoir nous concentrer sur les écrans web permettant les actions d'affichage, création, édition et suppression.

Vous remarquerez dans les extraits de code suivants que les champs " rowguid " et " ModifiedDate " n'apparaissent pas pour des raisons de visibilité et facilité de saisie. Pour ceux qui ont jetté un coup d'œil au niveau de la table " SalesTerritory ", vous aurez remarqué que les deux champs supprimés ont une valeur par défaut définie. Notre site web n'aura donc pas besoin d'assigner des valeurs pour ces champs, ils seront remplis automatiquement par la base de données.

VI-A. Implémentation de la grille de données

Avant de commencer, nous allons supprimer la vue " Index " générée automatiquement par Visual Studio lors de la création du projet ASP.NET MVC. Pour cela, supprimez le fichier " Views\Home\Index.aspx " du projet.

Nous allons à présent nous rendre dans le contrôleur " HomeController " (fichier " HomeController.cs ") afin que l'action " Index " renvoie la liste des territoires de vente provenant la base de données.

```
namespace MvcApplicationCRUD.Controllers
{
```

```
[HandleError]
public class HomeController : Controller
{
    private MvcApplicationCRUD.Models.AdventureWorksEntities _Context = new Models.AdventureWorksEntities();

    public ActionResult Index()
    {
        return View(_Context.SalesTerritory.ToList());
    }

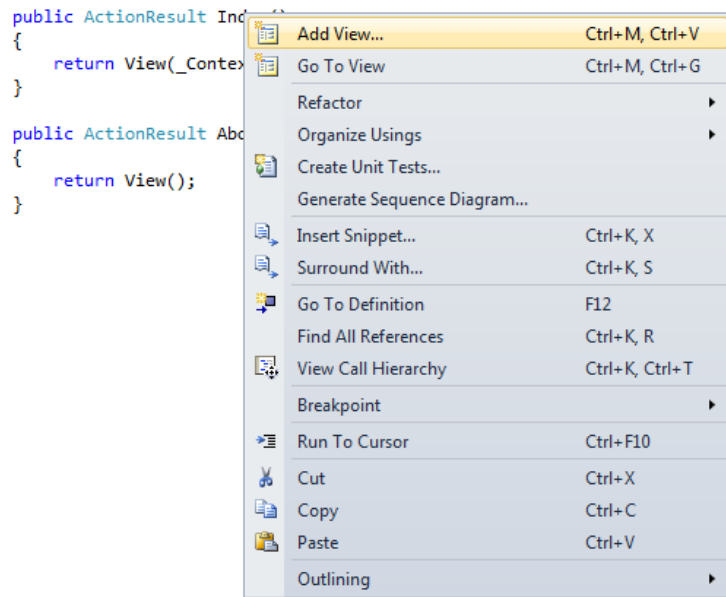
    public ActionResult About()
    {
        return View();
    }
}
```

La variable " _Context " a été ajoutée afin d'instancier un contexte pour l'accès à la base de données.

```
private MvcApplicationCRUD.Models.AdventureWorksEntities _Context = new Models.AdventureWorksEntities();
```

Quant à l'a méthode " Index ", elle renvoie simplement une liste de " SalesTerritory " à la vue.

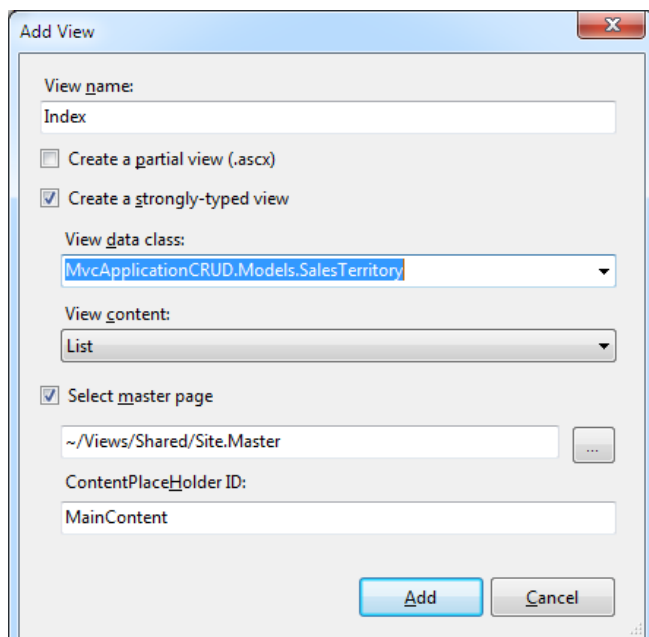
Il reste à présent à recréer une vue et pour cela, il suffit de faire un clic droit sur la méthode " Index " du contrôleur et de sélectionner l'action " Add View... ".



Dans la boîte de dialogue affichée, nous allons pouvoir préciser le type de vue que nous désirons en choisissant l'option " Create a strongly-typed view ". En sélectionnant cette option, nous allons pouvoir spécifier le type associé à cette fenêtre ainsi que sa représentation.

Nous allons donc renseigner la classe " MvcApplicationCRUD.Models.SalesTerritory " dans " View data class " et " List " dans " View Content ". Cela signifie que notre vue devra afficher sous forme d'un tableau des informations de la classe " SalesTerritory ".

Cliquez ensuite sur le bouton " Add " pour valider la création de la vue.



Si la classe " MvcApplicationCRUD.Models.SalesTerritory " n'apparait pas, il vous faut compiler votre projet en allant dans le menu " Build\Build Solution ".

Le code suivant est généré dans la vue " Index.aspx " :

```
<%@ Page Title="" Language="C#" MasterPageFile="~/Views/Shared/Site.Master" Inherits="System.Web.Mvc.ViewPage<IEnumerable<MvcApp.
<asp:Content ID="Content1" ContentPlaceHolderID="TitleContent" runat="server">
    Index
</asp:Content>

<asp:Content ID="Content2" ContentPlaceHolderID="MainContent" runat="server">

    <h2>Index</h2>

    <table>
        <tr>
            <th></th>
            <th>
                TerritoryID
            </th>
            <th>
                Name
            </th>
            <th>
                CountryRegionCode
            </th>
            <th>
                Group
            </th>
            <th>
                SalesYTD
            </th>
            <th>
                SalesLastYear
            </th>
            <th>
                CostYTD
            </th>
            <th>
                CostLastYear
            </th>
        </tr>

        <% foreach (var item in Model) { %>

            <tr>
                <td>
                    <%: Html.ActionLink("Edit", "Edit", new { id=item.TerritoryID }) %> |
                    <%: Html.ActionLink("Details", "Details", new { id=item.TerritoryID })%> |
                    <%: Html.ActionLink("Delete", "Delete", new { id=item.TerritoryID })%>
                </td>
                <td>
                    <%: item.TerritoryID %>
                </td>
                <td>
                    <%: item.Name %>
                </td>
                <td>
                    <%: item.CountryRegionCode %>
                </td>
                <td>
                    <%: item.Group %>
                </td>
                <td>
                    <%: String.Format("{0:F}", item.SalesYTD) %>
                </td>
                <td>
                    <%: String.Format("{0:F}", item.SalesLastYear) %>
                </td>
                <td>
                    <%: String.Format("{0:F}", item.CostYTD) %>
                </td>
                <td>
                    <%: String.Format("{0:F}", item.CostLastYear) %>
                </td>
            </tr>

        <% } %>

    </table>

    <p>
        <%: Html.ActionLink("Create New", "Create") %>
    </p>

</asp:Content>
```



Vous pouvez constater que Visual Studio a choisi la balise " table " pour l'affichage de nos données.

Démarrons le site web via le menu " Debug\Start Debugging " afin d'obtenir le résultat suivant :

TerritoryID	Name	CountryRegionCode	Group	SalesYTD	SalesLastYear	CostYTD	CostLastYear
1	Northwest	US	North America	5767341,98	3298694,49	0,00	0,00
2	Northeast	US	North America	3857163,63	3607148,94	0,00	0,00
3	Central	US	North America	4677108,27	3205014,08	0,00	0,00
4	Southwest	US	North America	8351296,74	5366575,71	0,00	0,00
5	Southeast	US	North America	2851419,04	3925071,43	0,00	0,00
6	Canada	CA	North America	6917270,88	5693988,86	0,00	0,00
7	France	FR	Europe	3899045,69	2396539,76	0,00	0,00
8	Germany	DE	Europe	2481039,18	1307949,79	0,00	0,00
9	Australia	AU	Pacific	1977474,81	2278548,98	0,00	0,00
10	United Kingdom	GB	Europe	3514865,91	1635823,40	0,00	0,00

VI-B. Implémentation de l'affichage détaillé

Lors de la création de la vue Index.aspx, vous avez peut-être pu constater que Visual Studio a généré dans le code de la page la ligne suivante :

```
<%: Html.ActionLink("Details", "Details", new { id=item.TerritoryID })%>
```

Cette ligne suppose qu'il existe une action " Details " dans notre contrôleur prenant en paramètre l'identifiant d'un territoire.

Cette méthode n'existant pas encore au sein de notre contrôleur, nous allons donc la créer. Rendez-vous dans le contrôleur " HomeController " et ajoutons l'action " Details " comme suit :

```
public ActionResult Details(int id)
{
    ...
}
```

Vous noterez que le nom du paramètre est exactement le même que la propriété " id " déclarée dans l'objet anonyme au niveau du code HTML.

```
new { id=item.TerritoryID }
```

Maintenant que l'action " Details " est créée, il nous reste plus qu'à retourner le territoire désiré sur base de l'identifiant passé en paramètre.

Pour cela, rien de plus simple avec Entity Framework, il suffit de récupérer l'objet " SalesTerritory " associé à l'identifiant passé en paramètre.

```
public ActionResult Details(int id)
{
    return View(_Context.SalesTerritory.Single(territory => territory.TerritoryID == id));
}
```

Maintenant que notre contrôleur est prêt, il ne manque plus que la vue " Details ".

Comme précédemment pour la vue des territoires de vente il suffit de faire un clic droit sur la méthode " Details " du contrôleur et de sélectionner l'action " Add View... ".

Nous allons donc renseigner la classe " MvcApplicationCRUD.Models.SalesTerritory " dans " View data class " et " Details " dans " View Content ". Cela signifie que notre vue devra afficher un formulaire permettant la visualisation en lecture seule d'un " SalesTerritory " donné.

Cliquez ensuite sur le bouton " Add " pour valider la création de la vue.

View name: Details

☐ Create a partial view (.ascx)

☒ Create a strongly-typed view

View data class: MvcApplicationCRUD.Models.SalesTerritory

View content: Details

☒ Select master page

~/Views/Shared/Site.Master

ContentPlaceHolder ID: MainContent

Add Cancel

Vous devriez avoir le code suivant dans la nouvelle vue " Details.aspx " :

```
<%@ Page Title="" Language="C#" MasterPageFile="~/Views/Shared/Site.Master" Inherits="System.Web.Mvc.ViewPage<MvcApplicationCRUD
<asp:Content ID="Content1" ContentPlaceHolderID="TitleContent" runat="server">
    Details
</asp:Content>

<asp:Content ID="Content2" ContentPlaceHolderID="MainContent" runat="server">

    <h2>Details</h2>

    <fieldset>
        <legend>Fields</legend>

        <div class="display-label">TerritoryID</div>
        <div class="display-field"><%: Model.TerritoryID %></div>

        <div class="display-label">Name</div>
        <div class="display-field"><%: Model.Name %></div>

        <div class="display-label">CountryRegionCode</div>
        <div class="display-field"><%: Model.CountryRegionCode %></div>

        <div class="display-label">Group</div>
        <div class="display-field"><%: Model.Group %></div>

        <div class="display-label">SalesYTD</div>
        <div class="display-field"><%: String.Format("{0:F}", Model.SalesYTD) %></div>

        <div class="display-label">SalesLastYear</div>
        <div class="display-field"><%: String.Format("{0:F}", Model.SalesLastYear) %></div>

        <div class="display-label">CostYTD</div>
        <div class="display-field"><%: String.Format("{0:F}", Model.CostYTD) %></div>

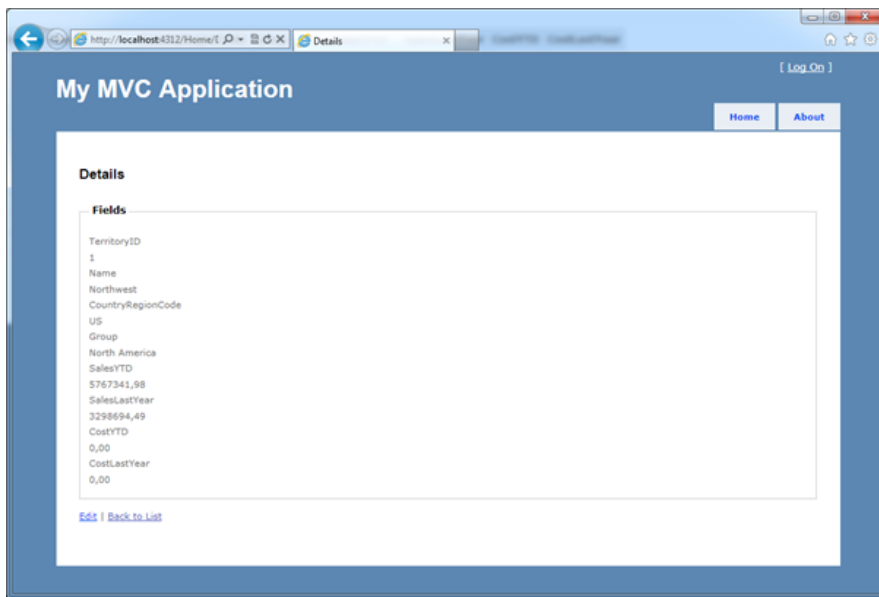
        <div class="display-label">CostLastYear</div>
        <div class="display-field"><%: String.Format("{0:F}", Model.CostLastYear) %></div>

    </fieldset>
    <p>

        <%: Html.ActionLink("Edit", "Edit", new { id=Model.TerritoryID }) %> |
        <%: Html.ActionLink("Back to List", "Index") %>

    </p>
</asp:Content>
```

Sur le site web, vous pouvez accéder à cette page en cliquant sur le lien " Details " d'un des territoires de vente.



VI-C. Implémentation de la création

Lors de la création de la vue " Index.aspx ", vous avez peut-être aussi pu constater que Visual Studio a généré dans le code de la page la ligne suivante :

```
<%: Html.ActionLink("Create New", "Create") %>
```

Cette ligne suppose qu'il existe une action " Create " dans notre contrôleur.

Cette méthode n'existant pas encore au sein de notre contrôleur, nous allons donc la créer. Rendez-vous dans le contrôleur " HomeController " et ajoutons l'action " Create " comme suit :

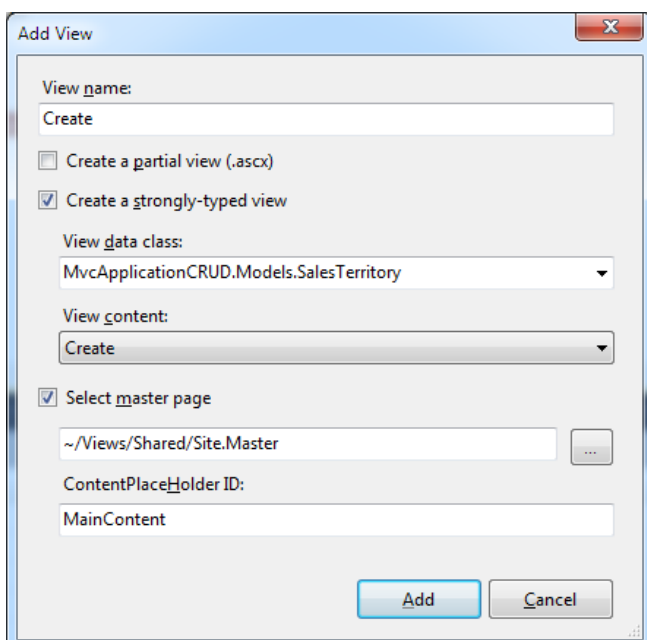
```
public ActionResult Create()
{
    return View();
}
```

Maintenant que notre contrôleur est prêt, il faut encore créer la vue " Create ".

Comme précédemment pour la vue des territoires de vente il suffit de faire un clic droit sur la méthode " Create " du contrôleur et de sélectionner l'action " Add View... ".

Nous allons donc renseigner la classe " MvcApplicationCRUD.Models.SalesTerritory " dans " View data class " et " Create " dans " View Content ". Cela signifie que notre vue devra afficher un formulaire permettant la création d'un nouveau " SalesTerritory ".

Cliquez ensuite sur le bouton " Add " pour valider la création de la vue.



Vous devriez avoir le code suivant dans la nouvelle vue " Create.aspx " :

```
<%@ Page Title="" Language="C#" MasterPageFile="~/Views/Shared/Site.Master" Inherits="System.Web.Mvc.ViewPage<MvcApplicationCRUD
    Models.SalesTerritory>" %>
<asp:Content ID="Content1" ContentPlaceHolderID="TitleContent" runat="server">
    Create
</asp:Content>
```

```

</asp:Content>

<asp:Content ID="Content2" ContentPlaceHolderID="MainContent" runat="server">

    <h2>Create</h2>

    <% using (Html.BeginForm()) {%>
        <%: Html.ValidationSummary(true) %>

        <fieldset>
            <legend>Fields</legend>

            <div class="editor-label">
                <%: Html.LabelFor(model => model.TerritoryID) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.TerritoryID) %>
                <%: Html.ValidationMessageFor(model => model.TerritoryID) %>
            </div>

            <div class="editor-label">
                <%: Html.LabelFor(model => model.Name) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.Name) %>
                <%: Html.ValidationMessageFor(model => model.Name) %>
            </div>

            <div class="editor-label">
                <%: Html.LabelFor(model => model.CountryRegionCode) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.CountryRegionCode) %>
                <%: Html.ValidationMessageFor(model => model.CountryRegionCode) %>
            </div>

            <div class="editor-label">
                <%: Html.LabelFor(model => model.Group) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.Group) %>
                <%: Html.ValidationMessageFor(model => model.Group) %>
            </div>

            <div class="editor-label">
                <%: Html.LabelFor(model => model.SalesYTD) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.SalesYTD) %>
                <%: Html.ValidationMessageFor(model => model.SalesYTD) %>
            </div>

            <div class="editor-label">
                <%: Html.LabelFor(model => model.SalesLastYear) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.SalesLastYear) %>
                <%: Html.ValidationMessageFor(model => model.SalesLastYear) %>
            </div>

            <div class="editor-label">
                <%: Html.LabelFor(model => model.CostYTD) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.CostYTD) %>
                <%: Html.ValidationMessageFor(model => model.CostYTD) %>
            </div>

            <div class="editor-label">
                <%: Html.LabelFor(model => model.CostLastYear) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.CostLastYear) %>
                <%: Html.ValidationMessageFor(model => model.CostLastYear) %>
            </div>

            <p>
                <input type="submit" value="Create" />
            </p>
        </fieldset>

    <% } %>

    <div>
        <%: Html.ActionLink("Back to List", "Index") %>
    </div>

</asp:Content>

```

Sur le site web, vous pouvez accéder à cette page en cliquant sur le lien " Create New " en bas de la grille des territoires de vente.

Malgré tout ce qui a été déjà fait, le travail n'est pas encore fini pour réellement créer un nouveau territoire de vente. Nous avons bien un formulaire pour la saisie du territoire de vente mais aucun mécanisme pour sa sauvegarde en base de données.

Si on regarde de plus près le code généré dans la vue " Create.aspx ", on peut remarquer qu'un bouton a été créé permettant de renvoyer le formulaire au serveur.

```
<input type="submit" value="Create" />
```

En HTML, cela signifie qu'une requête de type POST est renvoyée sur le serveur pour cette même page. C'est là qu'ASP.NET MVC va simplifier grandement les choses pour nous en s'attendant à trouver une action " Create " accessible en POST et prenant en paramètre... Un instance de la classe " SalesTerritory " étant donné que ce sont les informations que l'utilisateur a pu remplir au travers du formulaire de création.

Dans notre contrôleur, il suffit d'ajouter la méthode suivante :

```
[HttpPost()]
public ActionResult Create(MvcApplicationCRUD.Models.SalesTerritory territory)
{
}
```

L'attribut " HttpPost " qui décore notre méthode " Create " signifie que si on tente d'atteindre l'url via une requête HTTP de type POST, ce sera cette méthode qui sera appelée. Dans le cas d'une requête d'un autre type, comme GET, ce sera la première méthode " Create " qui sera appelée.

Il reste à présent à sauvegarder notre objet de type " SalesTerritory " en base de données via Entity Framework.

```
[HttpPost()]
public ActionResult Create(MvcApplicationCRUD.Models.SalesTerritory territory)
{
    _Context.SalesTerritory.AddObject(territory);
    _Context.SaveChanges();
    return RedirectToAction("Index");
}
```

VI-D. Implémentation de l'édition

Lors de la création de la vue " Index.aspx ", vous avez peut-être pu constater que Visual Studio a généré dans le code de la page la ligne suivante :

```
<%= Html.ActionLink("Edit", "Edit", new { id=item.TerritoryID }) %>
```

Cette ligne suppose qu'il existe une action " Edit " dans notre contrôleur prenant en paramètre l'identifiant d'un territoire.

Cette méthode n'existant pas encore au sein de notre contrôleur, nous allons donc la créer. Rendez-vous dans le contrôleur " HomeController " et ajoutons l'action " Edit " comme suit :

```
public ActionResult Edit(int id)
{
    ...
}
```

Vous noterez que le nom du paramètre est exactement le même que la propriété " id " déclarée dans l'objet anonyme au niveau du code HTML.

```
new { id=item.TerritoryID }
```

Maintenant que l'action " Edit " est créée, il nous reste plus qu'à retourner le territoire désiré sur base de l'identifiant passé en paramètre.

Pour cela, rien de plus simple avec Entity Framework, il suffit de récupérer l'objet " SalesTerritory " associé à l'identifiant passé en paramètre.

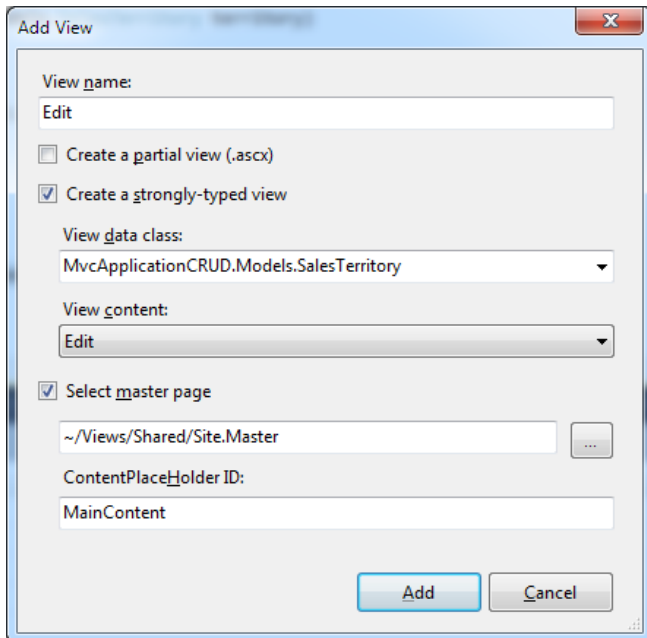
```
public ActionResult Edit(int id)
{
    return View(_Context.SalesTerritory.Single(territory => territory.TerritoryID == id));
}
```

Maintenant que notre contrôleur est prêt, il ne manque plus que la vue " Edit ".

Comme précédemment pour la vue des territoires de vente il suffit de faire un clic droit sur la méthode " Edit " du contrôleur et de sélectionner l'action " Add View... ".

Nous allons donc renseigner la classe " MvcApplicationCRUD.Models.SalesTerritory " dans " View data class " et " Edit " dans " View content ". Cela signifie que notre vue devra afficher un formulaire permettant la modification d'un " SalesTerritory " donné.

Cliquez ensuite sur le bouton " Add " pour valider la création de la vue.



Vous devriez avoir le code suivant dans la nouvelle vue " Details.aspx " :

```
<%@ Page Title="" Language="C#" MasterPageFile="~/Views/Shared/Site.Master" Inherits="System.Web.Mvc.ViewPage<MvcApplicationCRUD
Models.SalesTerritory>" %>
<asp:Content ID="Content1" ContentPlaceHolderID="TitleContent" runat="server">
    Edit
</asp:Content>
<asp:Content ID="Content2" ContentPlaceHolderID="MainContent" runat="server">
    <h2>Edit</h2>
    <% using (Html.BeginForm()) {%>
        <%: Html.ValidationSummary(true) %>
        <fieldset>
            <legend>Fields</legend>
            <div class="editor-label">
                <%: Html.LabelFor(model => model.TerritoryID) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.TerritoryID) %>
                <%: Html.ValidationMessageFor(model => model.TerritoryID) %>
            </div>
            <div class="editor-label">
                <%: Html.LabelFor(model => model.Name) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.Name) %>
                <%: Html.ValidationMessageFor(model => model.Name) %>
            </div>
            <div class="editor-label">
                <%: Html.LabelFor(model => model.CountryRegionCode) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.CountryRegionCode) %>
                <%: Html.ValidationMessageFor(model => model.CountryRegionCode) %>
            </div>
            <div class="editor-label">
                <%: Html.LabelFor(model => model.Group) %>
            </div>
            <div class="editor-field">
                <%: Html.TextBoxFor(model => model.Group) %>
            </div>
        </fieldset>
    <%>
</asp:Content>
```

```

        <%= Html.ValidationMessageFor(model => model.Group) %>
    </div>

    <div class="editor-label">
        <%= Html.LabelFor(model => model.SalesYTD) %>
    </div>
    <div class="editor-field">
        <%= Html.TextBoxFor(model => model.SalesYTD, String.Format("{0:F}", Model.SalesYTD)) %>
        <%= Html.ValidationMessageFor(model => model.SalesYTD) %>
    </div>

    <div class="editor-label">
        <%= Html.LabelFor(model => model.SalesLastYear) %>
    </div>
    <div class="editor-field">
        <%= Html.TextBoxFor(model => model.SalesLastYear, String.Format("{0:F}", Model.SalesLastYear)) %>
        <%= Html.ValidationMessageFor(model => model.SalesLastYear) %>
    </div>

    <div class="editor-label">
        <%= Html.LabelFor(model => model.CostYTD) %>
    </div>
    <div class="editor-field">
        <%= Html.TextBoxFor(model => model.CostYTD, String.Format("{0:F}", Model.CostYTD)) %>
        <%= Html.ValidationMessageFor(model => model.CostYTD) %>
    </div>

    <div class="editor-label">
        <%= Html.LabelFor(model => model.CostLastYear) %>
    </div>
    <div class="editor-field">
        <%= Html.TextBoxFor(model => model.CostLastYear, String.Format("{0:F}", Model.CostLastYear)) %>
        <%= Html.ValidationMessageFor(model => model.CostLastYear) %>
    </div>

    <p>
        <input type="submit" value="Save" />
    </p>
</fieldset>

<% } %>

<div>
    <%= Html.ActionLink("Back to List", "Index") %>
</div>

</asp:Content>

```

Sur le site web, vous pouvez accéder à cette page en cliquant sur le lien " Edit " d'un des territoires de vente.

Comme pour la création d'un territoire de vente, le travail n'est pas encore fini pour réellement modifier un territoire de vente. Nous avons bien un formulaire pour la saisie du territoire de vente mais aucun mécanisme pour sa sauvegarde en base de données.

Si on regarde de plus près le code généré dans la vue " Edit.aspx ", on peut remarquer qu'un bouton a été créé permettant de renvoyer le formulaire au serveur.

```
<input type="submit" value="Save" />
```

En HTML, cela signifie qu'une requête de type POST est renvoyée sur le serveur pour cette même page. C'est là qu'ASP.NET MVC va simplifier grandement les choses pour nous en s'attendant à trouver une action " Edit " accessible en POST et prenant en paramètre... Une instance de la classe " SalesTerritory " étant donné que ce sont les informations que l'utilisateur a pu remplir au travers du formulaire de création.

Dans notre contrôleur, il suffit d'ajouter la méthode suivante :

```
[HttpPost()]
public ActionResult Edit(MvcApplicationCRUD.Models.SalesTerritory territory)
{
}
}
```

L'attribut " HttpPost " qui décore notre méthode " Edit " signifie que si on tente d'atteindre l'url via une requête HTTP de type POST, ce sera cette méthode qui sera appelée. Dans le cas d'une requête d'un autre type, comme GET, ce sera la première méthode " Edit " qui sera appelée.

Il reste à présent à sauvegarder notre objet de type " SalesTerritory " en base de données via Entity Framework.

```
[HttpPost()]
public ActionResult Edit(MvcApplicationCRUD.Models.SalesTerritory territoryToUpdate)
{
    MvcApplicationCRUD.Models.SalesTerritory originalTerritory = _Context.SalesTerritory.Single(territory => territory.TerritoryID == territoryToUpdate.TerritoryID);
    originalTerritory.Name = territoryToUpdate.Name;
    originalTerritory.CountryRegionCode = territoryToUpdate.CountryRegionCode;
    originalTerritory.Group = territoryToUpdate.Group;
    originalTerritory.SalesYTD = territoryToUpdate.SalesYTD;
    originalTerritory.SalesLastYear = territoryToUpdate.SalesLastYear;
    originalTerritory.CostYTD = territoryToUpdate.CostYTD;
    originalTerritory.CostLastYear = territoryToUpdate.CostLastYear;
    originalTerritory.ModifiedDate = DateTime.Now;
    _Context.SaveChanges();
    return RedirectToAction("Index");
}
```

VI-E. Implémentation de la suppression

Lors de la création de la vue " Index.aspx ", vous avez peut-être pu constater que Visual Studio a généré dans le code de la page la ligne suivante :

```
<%: Html.ActionLink("Delete", "Delete", new { id=item.TerritoryID })%>
```

Cette ligne suppose qu'il existe une action " Delete " dans notre contrôleur prenant en paramètre l'identifiant d'un territoire.

Cette méthode n'existant pas encore au sein de notre contrôleur, nous allons donc la créer. Rendez-vous dans le contrôleur " HomeController " et ajoutons l'action " Delete " comme suit :

```
public ActionResult Delete(int id)
{
}
}
```

Vous noterez que le nom du paramètre est exactement le même que la propriété " id " déclarée dans l'objet anonyme au niveau du code HTML.

```
new { id=item.TerritoryID }
```

Maintenant que l'action " Delete " est créée, il nous reste plus qu'à retourner le territoire désiré sur base de l'identifiant passé en paramètre.

Pour cela, rien de plus simple avec Entity Framework, il suffit de récupérer l'objet " SalesTerritory " associé à l'identifiant passé en paramètre.

```
public ActionResult Delete(int id)
{
    return View(_Context.SalesTerritory.Single(territory => territory.TerritoryID == id));
}
```

Maintenant que notre contrôleur est prêt, il ne manque plus que la vue " Delete ".

Comme précédemment pour la vue des territoires de vente il suffit de faire un clic droit sur la méthode " Delete " du contrôleur et de sélectionner l'action " Add View... ".

Nous allons donc renseigner la classe " MvcApplicationCRUD.Models.SalesTerritory " dans " View data class " et " Delete " dans " View Content ". Cela signifie que notre vue devra afficher un formulaire permettant la modification d'un " SalesTerritory " donné.

Cliquez ensuite sur le bouton " Add " pour valider la création de la vue.

Vous devriez avoir le code suivant dans la nouvelle vue " Delete.aspx " :

```
<%@ Page Title="" Language="C#" MasterPageFile="~/Views/Shared/Site.Master" Inherits="System.Web.Mvc.ViewPage<MvcApplicationCRUD
<asp:Content ID="Content1" ContentPlaceHolderID="TitleContent" runat="server">
    Delete
</asp:Content>

<asp:Content ID="Content2" ContentPlaceHolderID="MainContent" runat="server">

    <h2>Delete</h2>

    <h3>Are you sure you want to delete this?</h3>
    <fieldset>
        <legend>Fields</legend>

        <div class="display-label">TerritoryID</div>
        <div class="display-field"><%: Model.TerritoryID %></div>

        <div class="display-label">Name</div>
        <div class="display-field"><%: Model.Name %></div>

        <div class="display-label">CountryRegionCode</div>
        <div class="display-field"><%: Model.CountryRegionCode %></div>

        <div class="display-label">Group</div>
        <div class="display-field"><%: Model.Group %></div>

        <div class="display-label">SalesYTD</div>
        <div class="display-field"><%: String.Format("{0:F}", Model.SalesYTD) %></div>

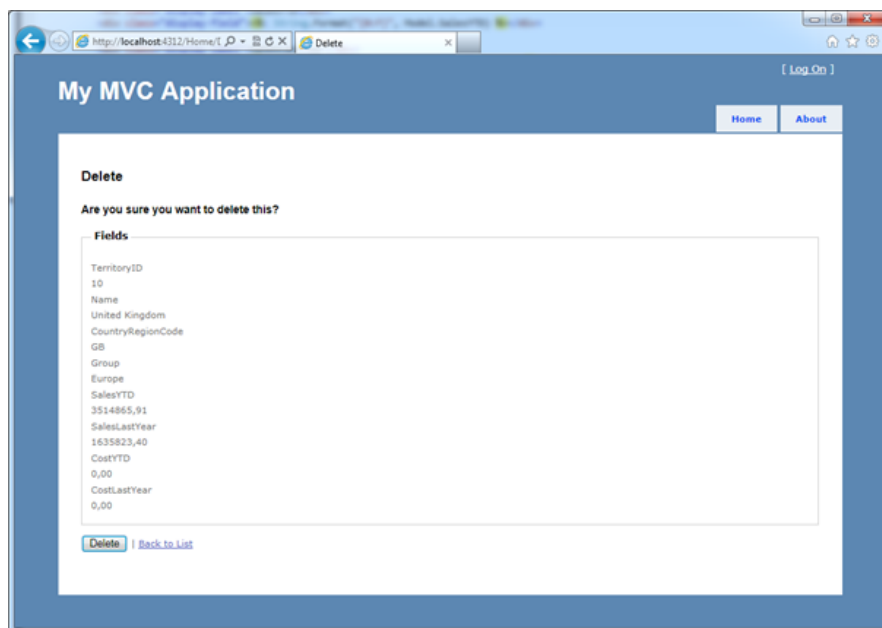
        <div class="display-label">SalesLastYear</div>
        <div class="display-field"><%: String.Format("{0:F}", Model.SalesLastYear) %></div>

        <div class="display-label">CostYTD</div>
        <div class="display-field"><%: String.Format("{0:F}", Model.CostYTD) %></div>

        <div class="display-label">CostLastYear</div>
        <div class="display-field"><%: String.Format("{0:F}", Model.CostLastYear) %></div>

    </fieldset>
    <% using (Html.BeginForm()) { %>
        <p>
            <input type="submit" value="Delete" /> |
            <%: Html.ActionLink("Back to List", "Index") %>
        </p>
    <% } %>
</asp:Content>
```

Sur le site web, vous pouvez accéder à cette page en cliquant sur le lien " Delete " d'un des territoires de vente.



Comme pour la création d'un territoire de vente, le travail n'est pas encore fini pour réellement supprimer un territoire de vente. Nous avons bien un formulaire qui affiche le détail de notre territoire de vente mais aucun mécanisme pour sa suppression en base de données.

Si on regarde de plus près le code généré dans la vue " Delete.aspx ", on peut remarquer qu'un bouton a été créé permettant de renvoyer le formulaire au serveur.

```
<input type="submit" value="Delete" />
```

En HTML, cela signifie qu'une requête de type POST est renvoyée sur le serveur pour cette même page. C'est là qu'ASP.NET MVC va simplifier grandement les choses pour nous en s'attendant à trouver une action " Delete " accessible en POST et prenant en paramètre l'identifiant qui se trouvent dans l'url.

Dans notre contrôleur, il suffit d'ajouter la méthode suivante :

```
[HttpPost()]
public ActionResult Delete(int id, FormCollection collection)
{
}
```

L'attribut " HttpPost " qui décore notre méthode " Delete " signifie que si on tente d'atteindre l'url via une requête HTTP de type POST, ce sera cette méthode qui sera appelée. Dans le cas d'une requête d'un autre type, comme GET, ce sera la première méthode " Edit " qui sera appelée.

Il reste à présent à supprimer notre objet de type " SalesTerritory " en base de données via Entity Framework.

```
[HttpPost()]
public ActionResult Delete(int id, FormCollection collection)
{
    MvcApplicationCRUD.Models.SalesTerritory territoryToDelete = _Context.SalesTerritory.Single(territory => territory.TerritoryID == id);
    _Context.SalesTerritory.DeleteObject(territoryToDelete);
    _Context.SaveChanges();
    return RedirectToAction("Index");
}
```

VII. Conclusion

Comme vous avez pu le voir tout au long de ce tutoriel, ASP.NET MVC couplé à ADO.NET Entity Framework offre aux développeurs une simplicité accrue quant aux opérations de type CRUD dans un site web grâce en outre à la génération automatique des formulaires au sein des vues ou encore à l'efficacité des contrôleurs avec leurs méthodes et paramètres automatiquement pris en charge par le framework.

Version PDF (Miroir) Version hors-ligne (Miroir)

Copyright © 2011 Jérôme Lambert. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts. Droits de diffusion permanents accordés à Developpez LLC.

Responsable de la rubrique Microsoft DotNET : le Service Publications - Contacter par email

Developpez.com
Nous contacter
Participez
Informations légales

Services
Forum Microsoft DotNET
Blogs
Hébergement

Partenaires
PlanetHoster
Ikoula

